Pixel Battle — Combat-Feel Polish (Sub-project A) Design Spec

Pixel Battle — Combat-Feel Polish (Sub-project A) — Design Spec

> **For agentic workers:** This is a *design spec*. The step-by-step implementation plan is generated separately via `superpowers:writing-plans`. Do not implement directly from this document.

- **Date:** 2026-05-22
- **Status:** Design approved (the user waived the spec-review gate and asked for autonomous execution through to completion)
- **Relationship:** Sub-project **A** of a two-part response to a round of combat feedback. Sub-project **B** (combat depth — Flash/teleport movement actions, a self-buff status-effect system, ranged-AI behaviour, faster movement) requires engine + action-space changes and an RL retrain; it is deferred and gets its own spec later.

1. Motivation

After the combat-animation overhaul (`docs/superpowers/specs/2026-05-21-pixel-battle-combat-animation-design.md`) shipped, the user reviewed the rendered output — reaction: “much better” — and gave a new round of feedback. Decomposed, the feedback splits into:

- **Sub-project A (this spec) — visual/feel polish, no RL retrain:**
  - Characters are slightly too large on screen.
  - Hits don’t read as impactful — combat “feels like swinging at air” (空砍).
  - Attacks fire when the opponent is out of range, playing a full swing animation that then resolves as a guaranteed out-of-range miss.
- **Sub-project B (deferred) — combat depth, needs retrain:** faster movement, Flash/teleport/dash actions, self-buffs (shield/tenacity), genuine ranged-character behaviour.

2. Goal

Make combat *feel* tighter and weightier — without retraining the RL policy — through smaller framing, elimination of out-of-range whiff animations, and a strong hitstop + recoil + screen-shake reaction when hits land.

3. Scope & Constraints

In scope — four changes: 1. Smaller on-screen characters. 2. Per-skill attack-range gate (no out-of-range whiff animations). 3. Engine-layer hitstop. 4. Bigger hit-reaction pose + screen shake.

Hard constraint: No RL retraining. No change to the RL observation vector, the action space, or the reward function. The trained checkpoint stays 100% valid.

Out of scope (→ Sub-project B): faster movement speed; Flash/teleport/dash movement actions; a self-buff (shield / tenacity / status-effect) system; ranged-AI spacing behaviour. These need action-space/engine changes and a retrain.

4. Current state (verified)

- **Attack range gating already exists.** `battle.py:_resolve_attack_hit` checks distance against `MELEE_RANGE` (110 px) or `SPECIAL_RANGE` (130 px) and emits a `MISS` (`reason: "out_of_range"`) if too far. Separately, `env.py:_apply_action` pre-gates attack actions with a single `ATTACK_GATE_RANGE = 145 px` — attacks issued beyond 145 px are a no-op. The **35 px gap** between that gate (145) and the melee hit range (110) is the "air-swing" zone: a melee attack issued at 111–145 px passes the gate, plays its full animation, then resolves as a guaranteed `out_of_range` miss.
- **Hit reactions already exist.** On a hit, `battle.py` sets the defender to `hit_stagger` for `STAGGER_MS` (300 ms, or a per-skill `stagger_ms`), applies knockback, and cancels the defender's in-progress attack. This spec makes the *visual* reaction punchier; the engine mechanics already work.
- **Character size** is set by `play.py:CAM_ZOOM = 1.7` (the camera crops a `WIDTH/CAM_ZOOM × HEIGHT/CAM_ZOOM` world region and upscales) plus the renderer `_STYLES` proportions.

5. The Four Changes

5.1 Smaller characters

Lower `CAM_ZOOM` in `play.py` from `1.7` to **`1.45`** (a slight zoom-out — characters shrink, more of the stage is visible). `CAM_VIEW_W` / `CAM_VIEW_H` derive from `CAM_ZOOM`, so they widen automatically. `1.45` is a starting value, tuned in the validation render.

`pixel_battle/tests/test_poses.py`'s visual-safety test has a `_MAX_HEIGHT` constant and a comment derived from the old `CAM_VIEW_H = 502`. With a lower zoom the camera view is *taller*, so the existing poses remain safely in frame — but the comment/derivation must be updated so it stays accurate (the constant may stay or loosen; it must not become wrong).

5.2 Per-skill attack-range gate

In `env.py:_apply_action`, replace the single `ATTACK_GATE_RANGE = 145` pre-fire gate with a **per-skill-type** gate: a melee attack action is gated at `MELEE_RANGE` (110 px), a special at `SPECIAL_RANGE` (130 px). An attack action issued outside the relevant range is a no-op exactly as today (no animation plays). Result: a character only commits to a swing when the opponent is within range it can actually hit — eliminating the doomed-whiff band. (Whiffs from the opponent moving away *during* the windup, or from the accuracy roll, still occur — those are intentional and read fine.)

The gate value per skill type is `MELEE_RANGE` / `SPECIAL_RANGE` (a small tolerance margin is a tunable, validated by the render). This does not touch the observation, action space, or reward — the agent still issues the same actions; more of them simply no-op when out of range.

5.3 Engine-layer hitstop

`Battle` gains a `_hitstop_remaining` integer counter (initialised 0).

- **Set on hit:** when a hit resolves in `_resolve_attack_hit`, set `_hitstop_remaining` to a freeze duration — `HITSTOP_TICKS` for a normal hit (~3 ticks ≈ 50 ms), `HITSTOP_TICKS_HEAVY` for a crit or ultimate (~6 ticks ≈ 100 ms). Heavy hits freeze longer so they read as weightier.
- **Consumed in `step()`:** at the very top of `Battle.step()`, if `_hitstop_remaining > 0`, decrement it and return immediately — the tick is frozen: no character moves, no new events, and `elapsed_ms` does **not** advance.

Because hitstop lives in the single simulation timeline, the renderer naturally draws the frozen frames and the audio events (placed by video-frame index) stay synced — no A/V bookkeeping needed.

Hitstop pauses the match clock (`elapsed_ms`), so it does not eat into the match timeout; it does lengthen the rendered video by the total freeze time (a few normal-hit freezes per match ≈ ~1 s — negligible against the ~20 s match and the 60 s render cap).

5.4 Bigger hit reaction

- **Recoil pose:** make `HIT_POSE` in `poses.py` more dramatic — torso recoils further back, arms flail wider, knees buckle more. The visual-safety test already exercises the `hit` pose, so it must continue to pass (feet planted, figure in frame) with the bigger pose.
- **Screen shake:** `play.py`'s camera adds a short, decaying random offset to the crop origin after a *heavy* hit event (crit or ultimate). A small magnitude that decays over a few frames. Normal hits do not shake (hitstop already covers them); shake is reserved for heavy hits so it stays meaningful.

6. Why no retrain

Every change is either renderer-side (`CAM_ZOOM`, screen shake, `HIT_POSE`) or an inert engine addition: - **Hitstop** inserts frozen ticks where `step()` is a no-op. The RL observation/action/reward are untouched;

during a frozen tick the agent's chosen action is simply not applied. The match's *decisions* are identical — only inert frozen ticks are interleaved. The trained policy runs unchanged. - **The attack gate** changes only which attack actions no-op. The observation, action space, and reward are untouched. The policy still issues the same actions; more no-op when out of range. It is mildly off-optimal (it learned the 145 gate) but still functional — and the existing approach-shaping reward already drives the agent to close distance.

No retrain is required. (Sub-project B will retrain — that is where the gate and movement speed get re-optimised.)

7. Risk

Tightening the attack gate could, in theory, make the AI look *passive* — an agent trained against the 145 gate might hover at 110–130 px issuing no-op attacks without closing. This is considered low-risk: the reward function already rewards closing distance (`approach` shaping) and landing damage, so the trained policy is already biased toward getting into hit range. The validation render (§9) will show this directly. If the fights do look passive, that is a signal for Sub-project B's retrain — or a quick widening of the gate margin.

8. Error handling

- `_hitstop_remaining` never goes negative (decrement only when `> 0`).
- The per-skill gate: an attack action whose skill type is unrecognised falls back to the melee gate (the stricter, safer default).
- Screen-shake magnitude decays to zero and is then inert (no offset applied).

9. Testing

New / updated unit tests: - **Hitstop:** a HIT event sets `_hitstop_remaining`; `Battle.step()` no-ops and decrements while frozen; `elapsed_ms` does not advance during a frozen tick; a crit/ult sets the heavy duration. - **Attack gate:** a melee attack action out of `MELEE_RANGE` is a no-op; a special attack action is gated at `SPECIAL_RANGE` (i.e. a special fires in the 110–130 band where a melee would not); in-range attacks still fire. - **Screen shake:** the shake offset is applied after a heavy hit and decays to zero. - **Visual safety:** the existing `test_all_poses_keep_feet_planted_and_in_frame` must still pass with the enlarged `HIT_POSE`. - All existing tests (312 at the start of this sub-project) stay green.

10. Validation

Render an `rl_play` episode and review: characters are smaller and better framed; hits land with a clear hitstop + recoil + (on heavy hits) screen shake; attacks are no longer thrown at out-of-range opponents.

Tune `CAM_ZOOM`, the hitstop tick counts, and the `HIT_POSE` angles from what the render shows.